

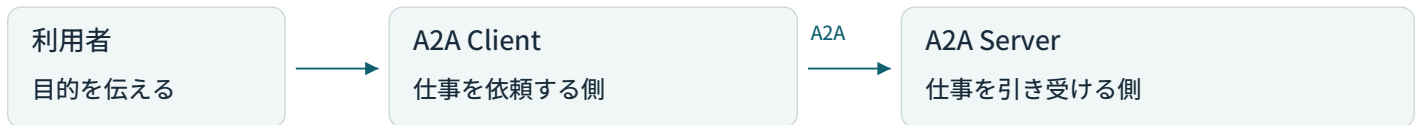
A2Aの全体像

Agent2Agent Protocol / 全体像から学ぶための地図

公式 v1.0 系を参照・2026-10-07 確認・A4 / 1枚

異なるAIエージェントが、共通のルールで仕事を依頼し、進捗や成果を交換する仕組み。
相手の内部のモデル・ツール・実装を共有せずに連携できる。

1 登場人物と役割



Clientはアプリでもよい。同じエージェントが、別の相手にはClientとして働くこともある。

2 基本の流れ：見つける → 頼む → 追う → 受け取る

- ① 相手を知る Agent Cardで、できる仕事・接続先・認証条件を確認する。
- ② 仕事を頼む Messageを送る（SendMessage）。返事はMessageまたはTask。
- ③ 進捗を追う Taskがあれば、状態を取得（GetTask）したり更新を受信する。
- ④ 成果を受け取る Taskの成果はArtifact。例：調査レポート、画像、JSONデータ。

例：旅行AgentがホテルAgentに「条件に合う宿を探して」と頼み、候補一覧を受け取る。

3 まず覚える6つの用語

Agent Card	相手の名刺。Skills（できる仕事）や対応機能を掲載。
Message	1回の発言・依頼・質問。会話を運ぶ。
Part	内容の部品。テキスト・ファイル・JSONなどを格納。
Task	状態を追える仕事。ID・状態・成果・履歴を持つ。
Artifact	仕事でできた成果物。内容は1つ以上のPartで表す。
contextId / taskId	関連する会話・仕事のまとめり / 特定の仕事を指すID。

4 Taskは「作業中」「追加情報待ち」「終了」を区別する

代表的な流れ：SUBMITTED（受付）→ WORKING（作業中）→ COMPLETED（完了）

途中で待つ：INPUT_REQUIRED（追加入力）/ AUTH_REQUIRED（追加認証）→ 対応後に継続

終了する：COMPLETED（完了）/ FAILED（失敗）/ CANCELED（取消）/ REJECTED（拒否）

終了したTaskは再開しない。修正依頼は同じcontextIdで新しいTaskにする。

※ 状態名はTASK_STATE_を省略。上記は例で、すべての段階を通る必要はない。

5 更新の受け取り方と、MCPとの関係

更新は3通り

Polling：Clientが繰り返し状態を聞く。
Streaming：接続中に進捗・成果を順次受信。
Push通知：Webhookで重要な更新を受信。

A2AとMCPは併用できる

A2A：独立したAgentに仕事を頼む。
MCP：Agentがツール・データを使う。
Streaming / Pushは相手の対応をCardで確認。

Railsで考えると：A2Aは外部との窓口。業務処理・DB・ジョブはRails側で実装する。

通信形式はJSON-RPC / gRPC / HTTP+JSON。認証・認可は既存方式とアプリ側で担う。

出典：a2a-protocol.org/latest/（各見出しから公式解説へリンク）

次に読む順：Core Concepts → Life of a Task → Specification（3章・4章）